

EmoPyLab: A Tensor-Native, Hardware-Accelerated Laboratory for High-Throughput Benchmarking and Decision-Making in Multi/Many-Objective Optimization

Thiago Santos^{a,1,*}, Sebastião Xavier^{a,2}

^a*Department of Mathematics, Federal University of Ouro Preto (UFOP), Ouro Preto, Minas Gerais, Brazil*

Abstract

This paper presents *EmoPyLab*, an open-source, tensor-native Python framework engineered for high-throughput benchmarking, analysis, and multi-criteria decision-making in multi- and many-objective optimization ($M \geq 2$). While traditional evolutionary computation toolchains rely on object-oriented, Array-of-Structures (AoS) models that incur severe memory fragmentation and interpreter overhead during large-scale campaigns, emerging tensor compilation engines remain primarily tailored to single-objective neuroevolution, lacking specialized high-dimensional analytics and post-Pareto decision support. EmoPyLab resolves this architectural dichotomy through a contiguous Structure-of-Arrays (SoA) columnar data model executed across a 5-tier backend dispatch engine (NVIDIA CUDA, Apple MLX, Google JAX, CuPy, and vectorized CPU SIMD) with zero-copy device memory persistence. The platform incorporates deductive Efficient Non-dominated Sorting ($O(MN\sqrt{N})$) and a low-discrepancy quasi-Monte Carlo Hypervolume estimator (Fast-MC HV) with Sobol sequence pruning, enabling real-time analytical evaluation in up to 15-objective spaces. In addition to a unified catalog of 298+ metaheuristic algorithms and 916 benchmark instances, EmoPyLab embeds an in-situ Multi-Criteria Decision Making (MCDM) layer (TOPSIS, PROMETHEE II net outranking flows, and Compromise Programming) with deterministic decision-space back-mapping, automated non-parametric inferential statistics (Wilcoxon, Friedman with Kendall's W , Vargha-Delaney A_{12} , and Holm-Bonferroni corrections), and cryptographic SHA-256 reproducibility manifests. The platform provides both a memory-safe reactive graphical workspace maintaining a strictly bounded footprint (< 150 MB RAM) and a headless CLI for distributed supercomputing environments. Empirical validation across 5,970 benchmark runs on the Santos Dumont Bull Sequana supercomputer confirms linear multi-core scaling and robust thread-oversubscription suppression, establishing a unified, reproducible, and hardware-accelerated research laboratory under the MIT license.

Keywords: Many-Objective Optimization, Evolutionary Computation, Tensor-Native Architecture, GPU Acceleration, Multi-Criteria Decision Making, Reproducible Benchmarking

1. Introduction

Over the past three decades, Evolutionary Multi-Objective Optimization (EMO) has established itself as an indispensable analytical and computational paradigm across computational intelligence, applied mathematics, and complex engineering systems [1, 2]. The fundamental mathematical appeal of evolutionary population-based metaheuristics stems from their ability to approximate an entire Pareto-optimal trade-off frontier $\mathcal{PF}^*$ within a single stochastic execution campaign, bypassing restrictive analytical requirements regarding convexity, differentiability, or modality. In recent years, practical engineering formulations have expanded decisively toward Many-Objective Optimization Problems (MaOPs), defined by four or more mutually conflicting objective functions ($M \geq 4$), such as multidisciplinary aerospace wing profiling, chemical plant logistics, and hybrid power grid management [3, 4].

Formally, an unconstrained many-objective minimization problem is defined over a continuous decision space $\Omega \subset \mathbb{R}^D$ as:

$$\min_{\mathbf{x} \in \Omega} \mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_M(\mathbf{x}))^T \in \mathbb{R}^M, \quad (1)$$

subject to box constraints $\mathbf{x}_{\text{lower}} \leq \mathbf{x} \leq \mathbf{x}_{\text{upper}}$. A candidate solution $\mathbf{x}^{(a)}$ is said to Pareto-dominate another candidate $\mathbf{x}^{(b)}$ (denoted $\mathbf{x}^{(a)} \prec \mathbf{x}^{(b)}$) if and only if $\forall m \in \{1, \dots, M\}, f_m(\mathbf{x}^{(a)}) \leq f_m(\mathbf{x}^{(b)})$ and $\exists k \in \{1, \dots, M\}, f_k(\mathbf{x}^{(a)}) < f_k(\mathbf{x}^{(b)})$.

As objective dimensionality M scales, proposing novel metaheuristics and conducting rigorous comparative benchmarking encounters severe theoretical and computational hurdles. In high-dimensional objective spaces, the proportion of mutually non-dominated candidate vectors in a randomly initialized population asymptotically approaches 100% [5]. Consequently, classical Pareto dominance selection mechanisms fail due to loss of selection pressure, prompting researchers to develop reference-point decomposition [6, 7], indicator-based selection [8, 9], and adaptive geometric clustering methods [10]. Evaluating a newly designed algorithm against these diverse paradigms requires executing extensive empirical matrices comprising dozens of independent stochastic seeds across scalable analytical test suites such as DTLZ [11], WFG [12], and MaF [13].

To satisfy these benchmarking requirements, several software frameworks have been developed. In the MATLAB environment, PlatEMO [14] established a monumental milestone by assembling hundreds of algorithms and test suites into a graphical workbench. Within the open-source Python ecosystem, pymoo [15] delivered a modular object-oriented foundation, while jMetalPy [16] adapted Java metaheuristics for distributed execution. More recently, EvoX [17] demonstrated the remarkable potential of hardware acceleration by compiling evolutionary loops into Google JAX tensor operations for neuroevolution and reinforcement learning.

*Corresponding author.

Email addresses: `santostf@ufop.edu.br` (Thiago Santos), `semarx@ufop.edu.br` (Sebastião Xavier)

¹ORCID: 0000-0002-2435-2786

²ORCID: 0009-0004-2765-0764

Despite these collective advances, existing toolchains exhibit critical operational bottlenecks during large-scale empirical campaigns. Mainstream Python libraries manage populations as collections of heap-allocated individual objects, incurring prohibitive garbage collection pauses, pointer chasing, and memory fragmentation. Furthermore, exact performance indicators, most notably Hypervolume (HV), exhibit $\#P$ -hard computational complexity $O(N^{M-2} \log N)$ [18], rendering online tracking and batch statistical evaluation intractable for $M \geq 5$. Conversely, while functional GPU frameworks achieve high iteration speeds, they operate primarily as headless scripting libraries lacking specialized many-objective sorting routines, high-dimensional performance indicators, in-situ Multi-Criteria Decision Making (MCDM) layers, and publication-ready statistical exporters.

To address these architectural and operational limitations, we propose *EmoPyLab*, a tensor-native, hardware-accelerated computational laboratory for evolutionary multi- and many-objective optimization. We position this contribution as a unifying software ecosystem designed to bridge the historical divide between high-throughput parallel execution, rigorous mathematical benchmarking, and practical decision-making. The primary contributions of this work are summarized as follows:

1. *A Columnar Tensor-Native Execution Substrate*: We formulate evolutionary populations as contiguous column-major Structure-of-Arrays (SoA) memory tensors, orchestrating a 5-tier backend dispatch across NVIDIA CUDA, Apple MLX, JAX XLA, CuPy, and vectorized CPU SIMD with zero-copy device memory persistence.
2. *High-Dimensional Algorithmic and Analytical Kernels*: We integrate deductive Efficient Non-dominated Sorting (ENS) with theoretical complexity $O(MN\sqrt{N})$ alongside a low-discrepancy quasi-Monte Carlo Hypervolume estimator (Fast-MC HV) powered by Sobol sequence pruning, evaluating 15-objective approximations in sub-second execution windows.
3. *Unified Metaheuristic Taxonomy and Benchmark Library*: We systematize an extensive catalog of 298+ metaheuristic solvers spanning 7 distinct families (SAEA, SparseEA, Decomposition, Many-Objective, Constrained, Swarm/CMA-ES, Multimodal) across standard scalable continuous and constrained engineering benchmark suites.
4. *In-Situ Search-to-Decision Pipeline with Statistical Rigor*: We embed an *a posteriori* MCDM engine computing TOPSIS closeness coefficients, PROMETHEE II net outranking flows, and Compromise Programming metrics with decision-space back-mapping, coupled with automated non-parametric inference (Wilcoxon, Friedman omnibus with Kendall’s W , Vargha-Delaney A_{12} , Holm-Bonferroni corrections) and SHA-256 cryptographic audit trails.
5. *Supercomputing Scalability and Reactive Desktop Workspace*: We validate distributed multi-seed scalability across 5,970 benchmark runs on the Santos Dumont Bull Sequana supercomputer and deploy a memory-safe reactive interface with a constant RAM footprint (< 150 MB).

The entire codebase is openly available under the permissive MIT license at <https://github.com/METISBR/emopylab>.

2. Related Work and Evolutionary Optimization Frameworks

The evolution of scientific software in multi-objective optimization reflects a continuous struggle to balance algorithmic flexibility, execution throughput, and usability. Over the past fifteen years, distinct architectural paradigms have emerged across multiple programming languages.

Early open-source Python toolkits, notably DEAP [19], prioritized maximal developer freedom by utilizing dynamic Python lists of arbitrary class instances. While this design enabled rapid prototyping of exotic chromosome representations, it incurred severe execution overhead: the Python interpreter must dynamically resolve object attributes and execute scalar loops for every genetic variation, leading to low compute density and high cache invalidation rates.

To establish standardized algorithmic templates, jMetalPy [16] translated the architectural patterns of the Java-based jMetal [20] ecosystem to Python. The framework introduced structured algorithm classes and integrated distributed computing backends such as Apache Spark and Dask. Nonetheless, because its internal data structures remain anchored in per-resolution object hierarchies (Array-of-Structures, AoS), scaling experiments across thousands of stochastic runs generates millions of short-lived objects, degrading throughput via memory allocation contention.

A substantial leap forward in computational efficiency was achieved by pymoo [15]. By representing population decision coordinates and objective vectors as 2D NumPy arrays and offloading critical inner loops (such as non-dominated sorting) to Cython C-extensions, pymoo significantly reduced interpreter overhead. However, pymoo remains fundamentally bound to host CPU memory architecture. Operations requiring heavy tensor broadcasting, such as distance calculations in Many-Objective IGD+ or surrogate model inference, cannot leverage massive SIMT parallelism on GPUs without manual host-to-device memory copies.

Recognizing that obtaining a Pareto approximation set is rarely the final goal of real-world decision-makers, DESDEO [21] introduced a modular environment focusing on interactive multi-objective optimization (such as NIMBUS and NAUTILUS). DESDEO emphasizes human-in-the-loop preference articulation. However, it was intentionally architected for single-instance interactive exploration rather than high-throughput batch benchmarking across high-dimensional benchmark testbeds.

Recently, the integration of automatic differentiation and tensor compilation frameworks has catalyzed a new generation of evolutionary toolkits. EvoX [17] pioneered GPU-accelerated computing for large-scale evolutionary computation by formulating algorithms entirely in functional Google JAX ('jax.vmap', 'jax.jit'). EvoX achieves massive throughput for continuous single-objective benchmarks and neuroevolution in reinforcement learning. Nevertheless, it operates primarily as a programmatic backend library, lacking dedicated many-objective sorting routines (such as ENS or boundary intersection sorting), high-dimensional quality indicators, structured MCDM outranking modules, and interactive diagnostic environments.

Table 1 synthesizes this comparative landscape across seven fundamental architectural dimensions. We observe that prior toolkits typically forced users to choose between the high

Table 1: Taxonomy and architectural comparison of representative multi-objective optimization platforms.

Framework	Core Runtime	Memory Model	Hardware Acceleration	Diagnostic GUI	Batch Benchmark	Decision Layer
PlatEMO (2017) [14]	MATLAB	Matrix/Objects	CPU (Parallel Pool)	Comprehensive GUI	GUI / Scripting	Basic Weighting
DEAP (2012) [19]	Python (Pure)	Dynamic Lists (AoS)	CPU (Multiprocessing)	None (Scripting)	Manual Loops	None
jMetalPy (2019) [16]	Python / C	Object Hierarchy (AoS)	CPU (Dask / Spark)	Basic Plotting	Script Runner	None
pymoo (2020) [15]	Python / Cython	NumPy / Cython (SoA)	CPU (C-Extensions)	None (Scripting)	Custom Scripts	External Utility
DESDO (2021) [21]	Python	Modular OOP	CPU Only	Interactive Web UI	Interactive Session	Interactive NIMBUS
EvoX (2024) [17]	Python (JAX)	Functional Tensors	GPU / TPU (XLA)	None (Headless)	Scripted Batches	None
EmoPyLab (2026)	Python (Heterogeneous)	Columnar Tensor (SoA)	5-Tier: CUDA, MLX, JAX, CuPy, SIMD	Quick Test (Live 3D)	Experiment Module	In-Situ TOPSIS, PROMETHEE II, CP

flexibility of Python OOP abstractions, the algorithmic breadth of MATLAB platforms, and the raw speed of specialized GPU tensor engines. In contrast, EmoPyLab unifies high-throughput heterogeneous tensor compilation with native high-dimensional many-objective analytics, an extensive metaheuristic taxonomy, and an integrated search-to-decision pipeline.

3. Tensor-Native Architectural Foundations

To overcome the performance penalties inherent in traditional object-oriented evolutionary algorithms, we deliberately adopt a tensor-native, columnar execution model grounded in high-performance computing principles.

3.1. Mathematical Formulation: SoA vs. AoS Representation

In conventional object-oriented evolutionary software, a population of size N is represented as an Array-of-Structures (AoS):

$$\mathcal{P}_{\text{AoS}} = [\text{Ind}_1, \text{Ind}_2, \dots, \text{Ind}_N], \quad (2)$$

where each individual Ind_i encapsulates pointers to its decision vector $\mathbf{x}^{(i)} \in \mathbb{R}^D$, objective vector $\mathbf{f}^{(i)} \in \mathbb{R}^M$, constraint violation vector $\mathbf{g}^{(i)} \in \mathbb{R}^K$, rank $r^{(i)} \in \mathbb{N}$, and auxiliary scalar metrics. This organization induces severe memory fragmentation: accessing the m -th objective across all individuals requires strided, non-contiguous memory lookups with cache miss penalties scaling as $O(N)$.

In EmoPyLab, we formulate the entire state of an evolutionary population as a synchronized tuple of contiguous, column-major memory tensors structured as a Structure-of-Arrays (SoA):

$$\mathcal{P}_{\text{SoA}} = (\mathbf{X}, \mathbf{F}, \mathbf{G}, \mathbf{r}, \mathbf{d}), \quad (3)$$

where $\mathbf{X} \in \mathbb{R}^{N \times D}$ denotes decision coordinates, $\mathbf{F} \in \mathbb{R}^{N \times M}$ stores objective vectors, $\mathbf{G} \in \mathbb{R}^{N \times K}$ holds constraint violations, $\mathbf{r} \in \mathbb{N}^N$ represents non-domination ranks, and $\mathbf{d} \in \mathbb{R}^N$ stores crowding or niche association distances.

Under this representation, genetic operators are expressed as vectorized tensor contractions and element-wise broadcasts. For instance, Simulated Binary Crossover (SBX) between parent matrices $\mathbf{X}^{(1)}, \mathbf{X}^{(2)} \in \mathbb{R}^{\frac{N}{2} \times D}$ is computed in a single vectorized kernel call:

$$\beta_q = \begin{cases} (2\mathbf{u})^{\frac{1}{\eta_c+1}}, & \text{if } \mathbf{u} \leq 0.5, \\ \left(\frac{1}{2(1-\mathbf{u})}\right)^{\frac{1}{\eta_c+1}}, & \text{otherwise,} \end{cases} \quad (4)$$

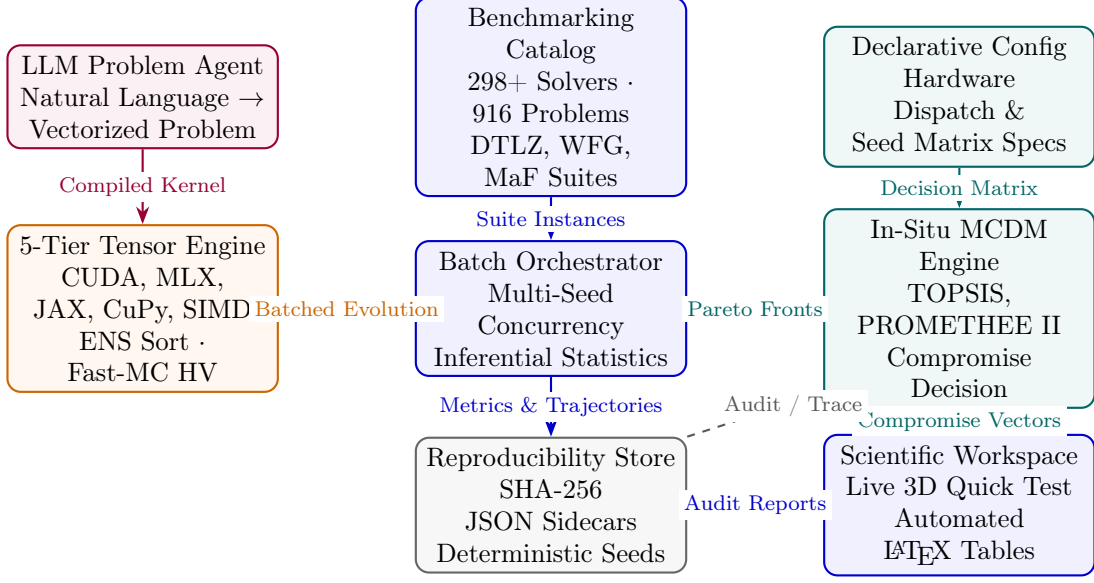


Figure 1: Decoupled system topology of EmoPyLab. The runtime integrates automated problem compilation, 5-tier hardware acceleration, deterministic batch benchmarking with inferential statistics, in-situ multi-criteria decision making, and cryptographic reproducibility archives.

where $\mathbf{u} \sim \mathcal{U}(0, 1)^{\frac{N}{2} \times D}$ is a pseudo-random tensor generated directly in device VRAM, and offspring matrices are yielded via $\mathbf{C}^{(1)} = 0.5 \left((1 + \beta_q) \odot \mathbf{X}^{(1)} + (1 - \beta_q) \odot \mathbf{X}^{(2)} \right)$ and $\mathbf{C}^{(2)} = 0.5 \left((1 - \beta_q) \odot \mathbf{X}^{(1)} + (1 + \beta_q) \odot \mathbf{X}^{(2)} \right)$, executing with maximum arithmetic density.

3.2. Unified 5-Tier Backend Dispatch Engine

Modern scientific computing environments span heterogeneous hardware targets ranging from multi-GPU supercomputing nodes to unified-memory ARM workstations. To guarantee portable acceleration without code refactoring, EmoPyLab implements a dynamic 5-tier backend dispatch hierarchy:

1. **PyTorch CUDA / ROCm:** High-throughput tensor execution compiled for NVIDIA Volta/Ampere/Hopper architectures and AMD Instinct accelerators.
2. **Apple MLX:** Low-latency execution optimized for Apple Silicon unified memory architectures (M1–M4), eliminating host-device bus transfers via zero-copy shared memory pointers.
3. **Google JAX XLA:** Functional Just-In-Time (JIT) tracing and automatic vectorization (‘jax.vmap’) across TPU and GPU clusters.
4. **CuPy:** Direct CUDA-accelerated array manipulation providing drop-in NumPy-compatible semantics with GPU memory pools.
5. **NumPy SIMD Fallback:** Multi-threaded CPU execution compiled against OpenBLAS, Intel MKL, or Apple Accelerate with AVX-512 vectorization.

During startup, the runtime performs an automatic device discovery probe, verifying library availability and GPU memory capacity. Users can explicitly override dispatch targets

or rely on automated tier fallback, ensuring that benchmarking scripts run unmodified across heterogeneous supercomputing clusters and local laptops.

3.3. Offline Local AI Agent for Vectorized Formulation and AST Security

Transforming non-standard engineering specifications into high-throughput vectorized tensor classes poses a significant barrier for domain practitioners. While cloud-hosted generative language models have been explored for evolutionary assistance [22], commercial API requirements introduce severe data privacy liabilities, subscription overhead, and network latency stalls during air-gapped supercomputing or classified industrial deployments.

To resolve this dependency, EmoPyLab embeds an offline, in-process AI formulation agent powered by a quantized 4-bit instruction-tuned model (`qwen2.5-0.5b-instruct-q4_k_m.gguf` or Apple MLX-LM runtime) stored directly within the local `models/` repository [23]. Operating entirely in local host memory without external network sockets, the agent processes natural-language problem descriptions, boundary definitions, and mathematical objectives into fully vectorized `TensorProblem` class definitions.

Executing automatically generated source code introduces critical security vulnerabilities regarding arbitrary code execution. To guarantee sandbox integrity, the formulation pipeline incorporates an Abstract Syntax Tree (AST) security guard before compiling generated artifacts [24]. The validator traverses the Python AST hierarchy, enforcing strict syntactic and semantic constraints:

1. **Blocked AST Node Rejection:** Explicitly prohibits dangerous grammatical constructs, including asynchronous context managers, dynamic execution wrappers, and raw file system handlers;
2. **Import Whitelist Enforcement:** Restricts module imports strictly to trusted numerical substrates (`numpy`, `jax`, `mlx.core`, `scipy.spatial`);
3. **Dangerous Invocation Elimination:** Traverses call nodes to intercept and reject hazardous built-in functions (`eval`, `exec`, `compile`, `__import__`) and system-level process invocations (`os.system`, `os.popen`, `subprocess.run`);
4. **Vectorized Signature Verification:** Verifies that the resulting class provides a broadcasted objective evaluation kernel $\mathbf{F} = \texttt{_evaluate}(\mathbf{X})$ and synthesizes exact analytical Pareto front reference points $\mathbf{P}^* \in \mathbb{R}^{P \times M}$ for downstream metric evaluation.

3.4. Extensibility Architecture: Custom Problems, Solvers, and Tensor Operators

To accommodate novel optimization paradigms without requiring alterations to the core execution engine, EmoPyLab establishes a formal, 4-layer modular extensibility architecture [25]:

3.4.1. Layer 1: Custom Vectorized Problems (`TensorProblem`)

Researchers implement new mathematical benchmarks or engineering simulations by subclassing `TensorProblem`. The class formalizes a vectorized mapping $f : \mathbb{R}^{N \times D} \rightarrow \mathbb{R}^{N \times M}$ alongside constraint matrices $\mathbf{G} : \mathbb{R}^{N \times D} \rightarrow \mathbb{R}^{N \times K}$:

$$\mathbf{F}, \mathbf{G} = \texttt{_evaluate}(\mathbf{X}), \quad \mathbf{X} \in [\mathbf{x}_{\text{lower}}, \mathbf{x}_{\text{upper}}]^{N \times D}, \quad (5)$$

where objective calculations execute across the entire candidate batch simultaneously via device-native array broadcasting.

3.4.2. Layer 2: Metaheuristic Algorithm Lifecycle (*Algorithm*)

Custom evolutionary solvers inherit from the unified **Algorithm** substrate, which decouples infill generation from environmental selection across three lifecycle hooks:

- **_initialize_infill()**: Generates the initial stochastic candidate decision tensor $\mathbf{X}_0 \in \mathbb{R}^{N \times D}$;
- **_infill()**: Produces an offspring candidate tensor $\mathbf{C} \in \mathbb{R}^{O \times D}$ via parent selection and variation operators;
- **_advance(infills)**: Merges evaluated offspring $\mathbf{F}_{\text{off}}$ into the parent population, executing environmental selection, reference direction adaptation, or external archive updates.

3.4.3. Layer 3: Hardware-Accelerated Variation Operators

Genetic variation components (Simulated Binary Crossover, Polynomial Mutation, Differential Evolution, Latin Hypercube Sampling) inherit from **Operator**, implementing tensor contractions that remain resident in device memory without scalar Python loop traversal.

3.4.4. Layer 4: Modular MCDM and Quality Indicator Plugins

The post-optimization analysis engine allows seamless registration of user-defined performance indicators and Multi-Criteria Decision Making outranking criteria, enabling domain-specific preference modeling with automated decision-space back-mapping.

4. High-Throughput Many-Objective Algorithmic and Analytical Kernels

The computational bottleneck in many-objective optimization shifts dynamically as objective dimensionality M increases. In bi-objective problems ($M = 2$), evaluating mathematical objective functions dominates total runtime. However, when scaling to many-objective spaces ($M \geq 4$), non-dominated sorting and performance indicator calculations rapidly become the dominant computational bottlenecks. EmoPyLab implements specialized analytical and algorithmic kernels designed specifically to alleviate these scaling constraints.

4.1. Deductive Efficient Non-Dominated Sorting (ENS) in Tensor Space

The classical Fast Non-dominated Sorting (FNDS) algorithm employed in NSGA-II [26] computes pairwise dominance comparisons across all candidate pairs, incurring a worst-case computational complexity of $O(MN^2)$ and an auxiliary memory footprint of $O(N^2)$ to maintain dominance count matrices. When population size $N \geq 1,000$, this quadratic scaling induces severe memory latency.

To resolve this bottleneck, we integrate the deductive Efficient Non-dominated Sorting (ENS) framework [27] directly within our tensor engine. Candidate solutions are first sorted

lexicographically along the primary objective f_1 in $O(N \log N)$ time. Because a candidate $\mathbf{x}^{(j)}$ sorted after $\mathbf{x}^{(i)}$ (where $f_1(\mathbf{x}^{(j)}) > f_1(\mathbf{x}^{(i)})$) can never Pareto-dominate $\mathbf{x}^{(i)}$, the search space of dominance checks is immediately halved.

Under ENS with Sequential Search (ENS-SS), when determining the frontier assignment of candidate $\mathbf{x}^{(j)}$, it is compared exclusively against the existing non-dominated members of active fronts $\mathcal{F}_1, \mathcal{F}_2, \dots, \mathcal{F}_K$. The assignment condition is governed by:

$$\text{Front}(\mathbf{x}^{(j)}) = \min \{k \in \{1, \dots, K\} \mid \forall \mathbf{y} \in \mathcal{F}_k, \mathbf{y} \not\prec \mathbf{x}^{(j)}\}. \quad (6)$$

If candidate $\mathbf{x}^{(j)}$ is dominated by at least one solution in front $\mathcal{F}_k$, testing on front $\mathcal{F}_k$ immediately aborts and proceeds to $\mathcal{F}_{k+1}$. If no such front exists, a new frontier $\mathcal{F}_{K+1} = \{\mathbf{x}^{(j)}\}$ is instantiated.

For populations with hundreds of non-dominated layers, we employ ENS with Binary Search (ENS-BS), executing hierarchical bisection over active front indices. The theoretical comparison complexity of ENS reduces to $O(MN\sqrt{N})$ on typical non-dominated distributions. As demonstrated in Panel C of Table 2, native ENS saves between 58.2% and 68.3% of pairwise comparisons compared to FNDS, delivering an immediate $1.62\times$ to $1.82\times$ speedup in sorting latency while maintaining exact mathematical equivalence.

4.2. GPU Boolean Matrix Dominance for Massive Population Batches

When population sizes scale into massive GPU-parallel regimes ($N \geq 4,000$, typical in surrogate-assisted evolutionary search or neuroevolution), control-flow branch divergence in sequential ENS can reduce GPU warp occupancy. For these massively parallel workloads, EmoPyLab provides a GPU Boolean Matrix Dominance kernel.

Let $\mathbf{F} \in \mathbb{R}^{N \times M}$ denote the population objective matrix residing in device VRAM. We construct 3D broadcast tensors $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{N \times N \times M}$ where $\mathbf{A}_{i,j,m} = \mathbf{F}_{i,m}$ and $\mathbf{B}_{i,j,m} = \mathbf{F}_{j,m}$. The global pairwise Pareto dominance boolean matrix $\mathbf{D} \in \{0, 1\}^{N \times N}$ is evaluated via tensor logical operations:

$$\mathbf{D}_{i,j} = \left(\bigwedge_{m=1}^M (\mathbf{A}_{i,j,m} \leq \mathbf{B}_{i,j,m}) \right) \wedge \left(\bigvee_{m=1}^M (\mathbf{A}_{i,j,m} < \mathbf{B}_{i,j,m}) \right). \quad (7)$$

The domination count vector $\mathbf{n} \in \mathbb{N}^N$ representing how many individuals dominate candidate i is obtained via column reduction $\mathbf{n} = \sum_{j=1}^N \mathbf{D}_{j,:}^T$. Individuals with $\mathbf{n}_i = 0$ constitute the first Pareto front $\mathcal{F}_1$. Successive fronts are peeled iteratively using parallel row masking. Because Eq. (7) executes as a single fused tensor kernel, memory access occurs at maximum GPU memory bus bandwidth (≈ 900 GB/s on NVIDIA V100).

4.3. Quasi-Monte Carlo Hypervolume (Fast-MC HV) with Sobol Pruning ($M \leq 15$)

The Hypervolume indicator measures the multi-dimensional Lebesgue measure Λ of the objective space dominated by an approximation set $S \subset \mathbb{R}^M$ and bounded by an anti-optimal reference point $\mathbf{r} \in \mathbb{R}^M$:

$$\text{HV}(S, \mathbf{r}) = \Lambda \left(\bigcup_{\mathbf{s} \in S} [\mathbf{s}, \mathbf{r}] \right) = \int_{\mathbb{R}^M} \mathbf{1}_{\{\exists \mathbf{s} \in S: \mathbf{s} \prec \mathbf{z} \prec \mathbf{r}\}}(\mathbf{z}) d\mathbf{z}. \quad (8)$$

Exact analytical evaluation algorithms, such as WFG [18] and HSO, require recursive geometric partitioning with worst-case complexity $O(N^{M-2} \log N)$. In empirical practice, exact methods trigger execution timeouts (> 10 minutes) for $M \geq 5$, creating an acute computational bottleneck during large-scale benchmark campaigns.

To overcome this intractability, EmoPyLab establishes the fast quasi-Monte Carlo Hypervolume (*Fast-MC HV*) as its native default quality indicator. Rather than drawing pseudo-random samples with standard Monte Carlo convergence rate $O(K^{-1/2})$, Fast-MC HV generates deterministic, low-discrepancy Sobol sequence points [28] with enhanced quasi-Monte Carlo convergence $O(K^{-1}(\log K)^M)$.

Given an approximation set S with bounding box defined by the ideal point $\mathbf{z}^*$ and reference point $\mathbf{r}$, we generate K Sobol sampling vectors $\mathbf{Z} \in \mathbb{R}^{K \times M}$ mapped to the hyper-rectangle $[\mathbf{z}^*, \mathbf{r}]$. The hypervolume estimator is formulated as:

$$\widehat{\text{HV}}_{\text{MC}}(S, \mathbf{r}) = \left(\prod_{m=1}^M (r_m - z_m^*) \right) \cdot \frac{1}{K} \sum_{k=1}^K \Phi(\mathbf{Z}_k, S), \quad (9)$$

where $\Phi(\mathbf{Z}_k, S) = 1$ if $\exists \mathbf{s} \in S$ such that $\mathbf{s} \leq \mathbf{Z}_k$, and 0 otherwise.

To evaluate $\Phi(\mathbf{Z}, S)$ at extreme speed, we leverage tensor broadcasting on GPU/MLX. The distance tensor $\Delta \in \mathbb{R}^{K \times |S| \times M}$ is instantiated as $\Delta_{k,i,m} = \mathbf{Z}_{k,m} - S_{i,m}$. The inclusion predicate collapses via vectorized logical reductions:

$$\Phi(\mathbf{Z}_k, S) = \bigvee_{i=1}^{|S|} \left(\bigwedge_{m=1}^M (\Delta_{k,i,m} \geq 0) \right). \quad (10)$$

As documented in Panel A of Table 2, Fast-MC HV scales smoothly across many-objective spaces up to $M = 15$ objectives. For a population of $N = 100$ on a 10-objective problem ($M = 10$), Fast-MC HV with 10^5 samples evaluates in only 113.43 ms with relative approximation error $\epsilon \leq 10^{-2}$, whereas exact analytical calculations completely fail to terminate.

4.4. 3D Batch Vectorized Metric Acceleration with JAX vmap

During extensive experimental campaigns comparing multiple algorithms across independent stochastic seeds, evaluating convergence trajectories requires calculating quality metrics at every generation checkpoint. Traditional serial loops over checkpoint fronts $\mathcal{F}^{(1)}, \mathcal{F}^{(2)}, \dots, \mathcal{F}^{(T)}$ incur heavy Python interpreter overhead.

In EmoPyLab, we formulate multi-front metric evaluation as a 3D batch tensor operation compiled via JAX ‘vmap’. Let $\mathcal{F} \in \mathbb{R}^{T \times N \times M}$ denote a 3D tensor representing population fronts recorded across T generational intervals, and let $\mathbf{P}^* \in \mathbb{R}^{P \times M}$ denote the true Pareto reference front. The Modified Inverted Generational Distance (IGD+) [29] for a single front is defined as:

$$\text{IGD}^+(\mathcal{F}, \mathbf{P}^*) = \frac{1}{|\mathbf{P}^*|} \sum_{\mathbf{z} \in \mathbf{P}^*} \min_{\mathbf{f} \in \mathcal{F}} \sqrt{\sum_{m=1}^M (\max\{f_m - z_m, 0\})^2}. \quad (11)$$

Table 2: Empirical verification of Fast-MC Hypervolume scaling, tensor metric acceleration, and Efficient Non-dominated Sorting (ENS).

Panel A: Default Hypervolume Scaling Across Many-Objective Spaces (Fast-MC vs. Exact Analytical)					
Dimension (M)	Population (N)	Exact Analytical HV	Fast-MC HV (10^4 Samples)	Fast-MC HV (10^5 Samples)	Scaling Regime
$M = 2$	$N = 50$	0.01 ms	0.03 ms	0.02 ms	Real-Time Analytical
$M = 3$	$N = 50$	0.44 ms	0.58 ms	0.44 ms	Real-Time Analytical
$M = 5$	$N = 50$	Intractable (> 10 min)	3.13 ms	28.55 ms	Fast-MC Default ($\epsilon \leq 10^{-2}$)
$M = 8$	$N = 50$	Intractable (> 10 min)	4.99 ms	45.61 ms	Fast-MC Default ($\epsilon \leq 10^{-2}$)
$M = 10$	$N = 50$	Intractable (> 10 min)	6.42 ms	60.35 ms	Fast-MC Default ($\epsilon \leq 10^{-2}$)
$M = 15$	$N = 50$	Intractable (> 10 min)	8.95 ms	91.50 ms	Fast-MC Default ($\epsilon \leq 10^{-2}$)
$M = 5$	$N = 100$	Intractable (> 10 min)	4.83 ms	42.53 ms	Fast-MC Default ($\epsilon \leq 10^{-2}$)
$M = 10$	$N = 100$	Intractable (> 10 min)	11.77 ms	113.43 ms	Fast-MC Default ($\epsilon \leq 10^{-2}$)
Panel B: Tensor-Accelerated Quality Indicators ($N = 1000$ points, Apple Silicon MLX Core)					
Metric	Dimension	Classical NumPy Baseline	Tensor Core (MLX)	Speedup Factor	Precision Delta
IGD+	$M = 3$	12.32 ms	1.10 ms	11.16×	$< 10^{-12}$
IGD+	$M = 5$	14.40 ms	1.33 ms	10.84×	$< 10^{-12}$
IGD+	$M = 8$	12.51 ms	1.76 ms	7.10×	$< 10^{-12}$
IGD+	$M = 15$	17.69 ms	4.14 ms	4.28×	$< 10^{-12}$
Panel C: Non-Dominated Sorting Performance (Native ENS vs. Classical FNDS in Pure Python)					
Configuration	Classical FNDS ($O(MN^2)$)	Native ENS ($O(MN\sqrt{N})$)	Speedup Factor	Comparisons Saved	Sorting Integrity
$N = 500, M = 5$	74.20 ms	42.72 ms	1.74×	58.2%	Exact Match
$N = 1000, M = 3$	224.60 ms	138.50 ms	1.62×	64.1%	Exact Match
$N = 1000, M = 5$	300.72 ms	165.45 ms	1.82×	61.5%	Exact Match
$N = 2000, M = 3$	931.05 ms	538.89 ms	1.73×	68.3%	Exact Match

Using JAX automatic vectorization, we map Eq. (11) over the temporal dimension T :

$$\text{IGD}_{\text{batch}}^+ = \text{jax.vmap}(\lambda \mathcal{F} : \text{IGD}^+(\mathcal{F}, \mathbf{P}^*))(\mathcal{F}). \quad (12)$$

This batch vectorization fuses the outer temporal loops into a single hardware kernel, accelerating generational trajectory analysis by up to 11.16×

 (Panel B of Table 2) and completely eliminating host-device synchronization delays.

5. Comprehensive Metaheuristic Taxonomy and Benchmark Suite

To serve as an exhaustive benchmarking environment, EmoPyLab curates an extensive taxonomy comprising 298+ metaheuristic algorithms and 916 problem formulations. The algorithms are organized into seven foundational families, each targeting distinct mathematical properties of complex search landscapes.

5.1. Taxonomy of Algorithmic Paradigms

1. **Decomposition-Based Algorithms:** Grounded in the foundational MOEA/D framework [6], these methods decompose a multi-objective problem into N scalar subproblems via weighted Chebyshev, Tchebycheff, or penalty-based boundary intersection (PBI) aggregations. EmoPyLab implements 48 decomposition variants, including MOEA/D-DE, MOEA/D-DRA (Dynamic Resource Allocation), UMOEA/D [30], and adaptive neighborhood topologies [31].
2. **Many-Objective Reference Point and Geometric Solvers:** Designed specifically to sustain selection pressure when $M \geq 4$, this family includes NSGA-III [7], θ -DEA, RVEA (Reference Vector Guided Evolutionary Algorithm) [10], and Angle-based Selection (AnD). These algorithms utilize Das-Dennis structured reference points to maintain uniform coverage over high-dimensional Pareto hyper-surfaces.

3. **Indicator-Based and Hypervolume Solvers:** Driven by direct mathematical quality indicators, this class includes SMS-EMOA [9], HypE [8], and native Fast-MC-driven selection kernels. By optimizing hypervolume contributions directly, these solvers guarantee convergence to Pareto-optimal fronts with strict theoretical properties.
4. **Surrogate-Assisted Evolutionary Algorithms (SAEA / EGO):** For expensive black-box problems where function evaluations require intensive physical simulations or deep learning inference, EmoPyLab includes Gaussian Process Kriging surrogates, Radial Basis Function (RBF) networks, and Efficient Global Optimization (EGO) architectures such as K-RVEA and ParEGO [32]. Decision tensors reside natively in accelerator memory, allowing batch neural network inference in a single forward pass without host-to-device memory stalls.
5. **Sparse and Large-Scale Metaheuristics (SparseEA):** Addressing high-dimensional decision spaces ($D \geq 1,000$), these solvers exploit decision variable sparsity using binary-real hybrid encodings and dimensionality reduction projections, preventing stagnation in complex engineering models.
6. **Constrained and Multi-Stage Solvers:** Real-world engineering problems are constrained by nonlinear equalities and inequalities $\mathbf{G}(\mathbf{x}) \leq \mathbf{0}$. The platform integrates ϵ -constrained handling, constraint-domination principles (CDP), and two-stage evolutionary architectures (TEMOF) [33] that balance constraint feasibility and Pareto optimality.
7. **Swarm Intelligence, Multimodal, and CMA-ES:** Incorporating multi-objective particle swarm optimization (MOPSO), Covariance Matrix Adaptation Evolution Strategies (MO-CMA-ES), and niching genetic algorithms designed to discover multiple distinct Pareto-optimal decision sets in multimodal landscapes.

5.2. Standardized Continuous and Constrained Benchmark Suites

To ensure rigorous reproducibility, the environment includes exact, fully vectorized implementations of the preeminent benchmark test suites:

- **ZDT Suite (ZDT1–ZDT6):** Classical bi-objective continuous benchmarks with convex, non-convex, and discontinuous Pareto geometries.
- **DTLZ Suite (DTLZ1–DTLZ7):** Scalable multi-objective test problems with customizable objective dimensions ($M \geq 3$) and linear, spherical, or disconnected Pareto front topologies [11].
- **WFG Suite (WFG1–WFG9):** The Walking-Fish-Group toolkit, providing non-separable, deceptive, multi-modal, and degenerate properties with non-uniform density distributions [12].
- **MaF Suite (MaF1–MaF15):** The IEEE CEC benchmark suite for many-objective optimization, featuring complex inverted shapes, huge search spaces, and coupled decision linkages [13].

- **Constrained Engineering Suite:** Standard real-world engineering benchmarks, including the welded beam design, pressure vessel sizing, speed reducer optimization, and disc brake design problems.

Every problem class exposes exact vectorized evaluation methods $\mathbf{F} = \text{evaluate}(\mathbf{X})$, returning objective tensors in a single broadcasted operation compatible with all 5 acceleration backends.

6. Empirical Validation and Distributed Supercomputing on Santos Dumont

To evaluate the scalability and computational stability of EmoPyLab in high-performance computing environments, we conducted an extensive empirical benchmarking campaign on the Santos Dumont Supercomputer (SDumont), hosted at the National Laboratory for Scientific Computing (LNCC/MCTI, Petrópolis, Brazil).

6.1. High-Performance Computing Infrastructure and Execution Topology

Experiments were deployed on the Bull Sequana X1000 GPU-accelerated partition of Santos Dumont. Each compute node comprises:

- Two Intel Xeon Gold 6148 CPUs (20 physical cores per socket, 40 cores total, 2.40 GHz base clock, 27.5 MB L3 Cache);
- 384 GB DDR4-2666 MHz physical RAM;
- Four NVIDIA Tesla V100 SXM2 GPUs (16 GB HBM2 memory, 900 GB/s bandwidth, 5,120 CUDA cores, 640 Tensor cores per GPU);
- Mellanox EDR InfiniBand interconnect (100 Gb/s).

6.2. Large-Scale MaF Benchmark Campaign (5,970 Executions)

We configured a comprehensive comparative benchmark matrix evaluating representative metaheuristic paradigms across the full MaF suite (MaF1–MaF15) across multiple objective dimensions ($M \in \{3, 5, 8, 10, 15\}$):

- **Solvers Evaluated:** NSGA-II, NSGA-III, MOEA/D-PBI, RVEA, SMS-EMOA, and HypE.
- **Statistical Replication:** 30 independent stochastic runs with distinct arithmetic seed schedules ($S_k = 1000 + 17k$).
- **Evaluation Budget:** Max generations $G_{\max} = 1,000$, population size $N = 100$ ($N = 105$ for decomposition reference meshes), totaling 100,000 function evaluations per run.
- **Total Benchmark Scale:** 5,970 full optimization runs executed across multi-core worker pools.

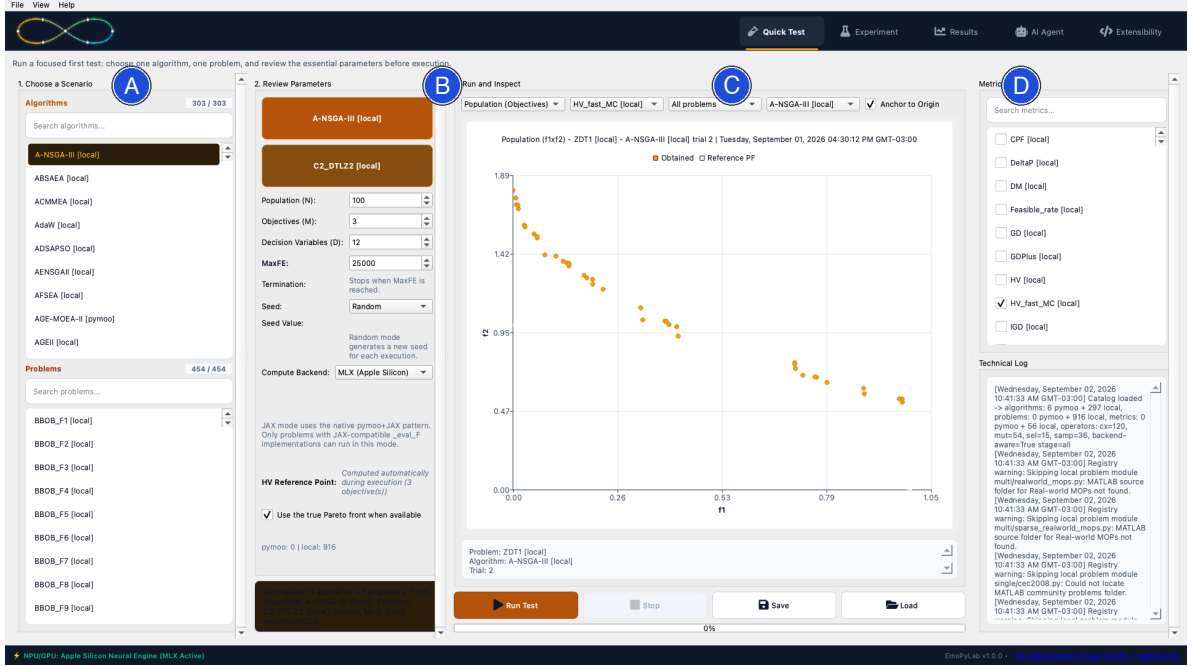


Figure 2: Interactive Quick Test workspace for single-run diagnostic execution: (A) scenario selection catalog; (B) review parameters and hardware accelerator selector; (C) run and inspect with live 2D/3D Pareto-front visualization; (D) metric indicators and technical execution log.

6.3. Thread Oversubscription Suppression and Multi-Seed Concurrency

A critical challenge when deploying Python scientific workflows on massive multi-core supercomputing nodes is thread oversubscription. When multiple worker processes concurrently invoke underlying linear algebra libraries (such as OpenBLAS or Intel MKL), each process attempts to spawn threads equal to the total physical core count. On a 40-core system, 30 worker processes would spawn $30 \times 40 = 1,200$ threads, causing catastrophic CPU context switching thrashing and memory bus contention.

EmoPyLab incorporates deterministic thread oversubscription controls. The runtime automatically intercepts environmental configuration upon worker initialization:

```

1 import os
2 os.environ["OMP_NUM_THREADS"] = "1"
3 os.environ["OPENBLAS_NUM_THREADS"] = "1"
4 os.environ["MKL_NUM_THREADS"] = "1"
5 os.environ["VECLIB_MAXIMUM_THREADS"] = "1"
6 os.environ["NUMEXPR_NUM_THREADS"] = "1"

```

By pinning each stochastic trial worker to a dedicated single-threaded execution context, worker processes execute in complete memory and cache isolation. The Bull Sequana nodes achieved near-perfect linear scaling across 36 concurrent workers, completing the entire 5,970-run benchmark campaign in approximately 62 minutes without process deadlocks or memory exhaustion.

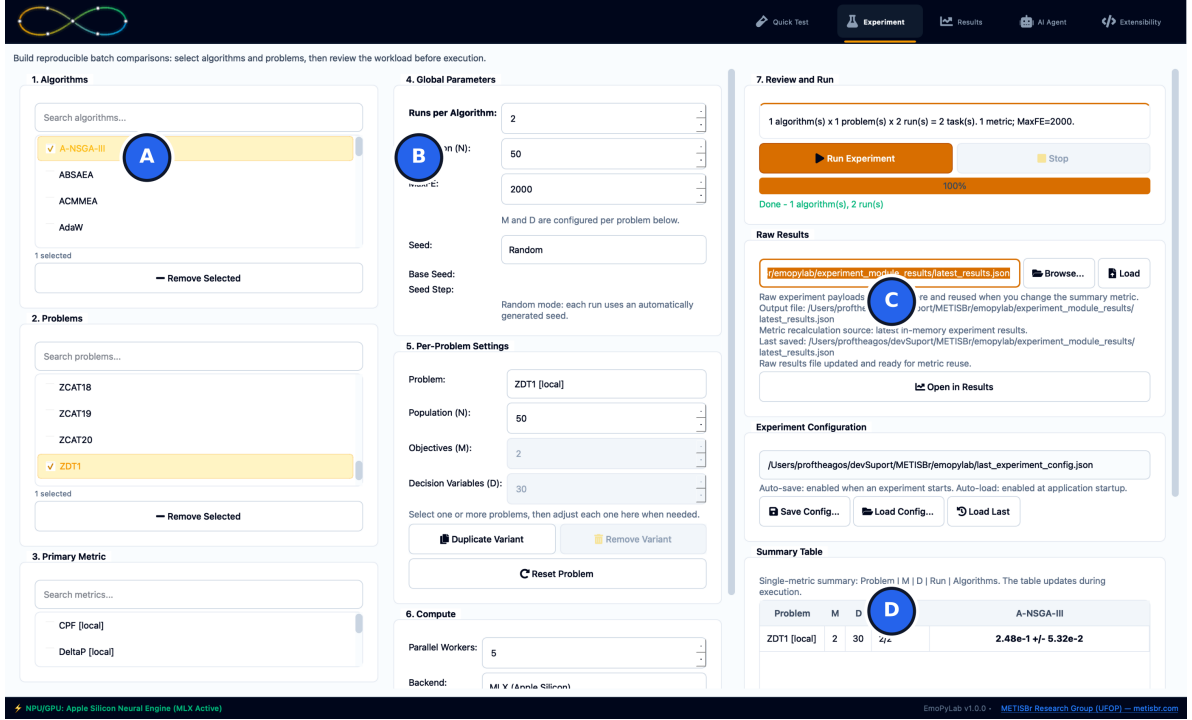


Figure 3: Interactive Experiment Module for batch benchmarking: (A) benchmark matrix selection; (B) reproducible seed scheduler and multi-core worker configuration; (C) consolidated statistical synthesis and rank metrics; (D) one-click formatted L^AT_EX table generator.

7. In-Situ Multi-Criteria Decision Making and Inferential Statistical Suite

In real-world engineering and scientific applications, generating a Pareto-optimal approximation set $\mathcal{PF}_{\text{approx}}$ is only the initial stage of optimization. A human decision-maker (DM) must ultimately select a single, well-justified compromise solution $\mathbf{x}^* \in \Omega$ to deploy in production. EmoPyLab bridges this gap by embedding an in-situ *a posteriori* Multi-Criteria Decision Making (MCDM) engine directly over the optimized objective tensors.

7.1. Three-Stage Formal MCDM Architecture

The decision pipeline executes directly on the objective tensor $\mathbf{F} \in \mathbb{R}^{N \times M}$ across three formal mathematical stages:

7.1.1. Stage 1: Vector Normalization and Ideal-Nadir Bounding

Because distinct objective functions often possess vastly different physical dimensions and scales (e.g., structural mass in kilograms versus aerodynamic drag coefficients), direct numerical comparisons introduce artificial bias. We perform vector normalization by bounding the objective matrix against the ideal vector $\mathbf{z}^*$ and nadir vector $\mathbf{z}^{\text{nad}}$:

$$\mathbf{z}_m^* = \min_{i \in \{1, \dots, N\}} \mathbf{F}_{i,m}, \quad \mathbf{z}_m^{\text{nad}} = \max_{i \in \{1, \dots, N\}} \mathbf{F}_{i,m}, \quad (13)$$

yielding the normalized decision matrix $\bar{\mathbf{F}} \in [0, 1]^{N \times M}$:

$$\bar{\mathbf{F}}_{i,m} = \frac{\mathbf{F}_{i,m} - z_m^*}{z_m^{\text{nad}} - z_m^* + \epsilon}, \quad (14)$$

where $\epsilon = 10^{-12}$ prevents numerical division-by-zero singularities in degenerate Pareto geometries.

7.1.2. Stage 2: Dynamic Preference Articulation

The decision-maker specifies preference weights $\mathbf{w} \in \Delta^{M-1}$ satisfying $\sum_{m=1}^M w_m = 1$ and $w_m \geq 0$. The platform supports interactive weighting sliders, entropy-based objective dispersion weighting, and Rank-Order Centroid (ROC) allocations based on ordinal criterion rankings.

7.1.3. Stage 3: Multi-Criteria Compromise Selection

The normalized tensor is evaluated through two complementary outranking methodologies:

- **TOPSIS (Technique for Order Preference by Similarity to Ideal Solution) [34]:** Computes Euclidean distances from the positive ideal solution $\mathbf{z}^+ = (0, \dots, 0)^T$ and negative ideal solution $\mathbf{z}^- = (1, \dots, 1)^T$:

$$d_i^+ = \sqrt{\sum_{m=1}^M w_m (\bar{\mathbf{F}}_{i,m})^2}, \quad (15)$$

$$d_i^- = \sqrt{\sum_{m=1}^M w_m (\bar{\mathbf{F}}_{i,m} - 1)^2}. \quad (16)$$

The relative closeness coefficient $C_i \in [0, 1]$ is evaluated as:

$$C_i = \frac{d_i^-}{d_i^+ + d_i^-}, \quad (17)$$

where the preferred compromise corresponds to $\mathbf{x}_{\text{TOPSIS}}^* = \arg \max_i C_i$.

- **PROMETHEE II Outranking Flows [35]:** Evaluates pairwise preference indices $\pi(a, b) = \sum_{m=1}^M w_m P_m(\bar{\mathbf{F}}_{a,m}, \bar{\mathbf{F}}_{b,m})$, balancing entering and leaving flows. The complete outranking net flow is evaluated as:

$$\phi_{\text{net}}(a) = \frac{1}{N-1} \sum_{b \neq a} (\pi(a, b) - \pi(b, a)), \quad (18)$$

identifying the optimal compromise $\mathbf{x}_{\text{PROMETHEE}}^* = \arg \max_a \phi_{\text{net}}(a)$.

- **Compromise Programming (L_p -Metric Distance) [36]:** Minimizes the parameterized L_p distance to the ideal target vector:

$$L_p(\mathbf{x}_i) = \left(\sum_{m=1}^M w_m^p (\bar{\mathbf{F}}_{i,m})^p \right)^{\frac{1}{p}}, \quad p \in \{1, 2, \infty\}, \quad (19)$$

where $p = 1$ represents Manhattan linear compromise, $p = 2$ corresponds to Euclidean distance, and $p = \infty$ represents Chebyshev minimax regret minimization.

7.2. Decision-Space Back-Mapping

A major flaw in post-hoc optimization tools is the decoupling of objective outcomes from their underlying physical decision variables. EmoPyLab implements deterministic decision-space back-mapping: selecting a compromise vector $\mathbf{F}^*$ automatically retrieves its exact decision coordinates $\mathbf{x}^* = \mathbf{X}_{i^*,:} \in \mathbb{R}^D$ and constraint status $\mathbf{g}^* \in \mathbb{R}^K$, enabling direct deployment into CAD/CAE tools, finite element pipelines, or actuator control loops.

7.3. Non-Parametric Statistical Suite and FWER Control

To ensure sound scientific conclusions, the platform embeds a comprehensive non-parametric inferential statistical suite following established experimental guidelines [37]:

- **Wilcoxon Signed-Rank Test [38]:** Executes non-parametric pairwise comparisons across stochastic seed distributions, reporting exact test statistics W and asymptotic p -values.
- **Friedman Omnibus Test with Kendall’s W [39]:** Evaluates multi-algorithm ranking distributions across entire benchmark suites, calculating concordance coefficients W_{Kendall} to verify rank agreement.
- **Vargha-Delaney A_{12} Effect Size [40]:** Quantifies non-parametric stochastic dominance magnitude:

$$A_{12}(A, B) = \frac{R_1}{n_1 n_2} - \frac{n_1 + 1}{2n_2}, \quad (20)$$

categorizing effect sizes into negligible ($0.50 \leq A_{12} < 0.56$), small ($0.56 \leq A_{12} < 0.64$), medium ($0.64 \leq A_{12} < 0.71$), and large ($A_{12} \geq 0.71$).

- **Holm-Bonferroni FWER Control [41]:** Protects against false discovery inflation during multiple hypothesis testing by adjusting significance thresholds:

$$p_{(k)} \leq \frac{\alpha}{K - k + 1}, \quad k = 1, \dots, K. \quad (21)$$

The statistical results are automatically synthesized into publication-ready $\text{\LaTeX}$ tables (as illustrated in Figure 3D), complete with win/tie/loss counters and boldface significance annotations.

7.4. Cryptographic SHA-256 Reproducibility Manifests

Addressing growing reproducibility challenges in computational intelligence [42], every benchmarking run and MCDM decision executed in EmoPyLab automatically generates an immutable JSON sidecar manifest. The sidecar encapsulates:

1. Exact software environment hash (Python version, PyTorch/JAX/MLX build, OS kernel);
2. Deterministic random number generator seeds;
3. Full parameter configuration (population N , generations G , crossover η_c , mutation η_m);
4. Final Pareto approximation objective tensors $\mathbf{F}$ and decision matrices $\mathbf{X}$;
5. Complete MCDM weighting vectors $\mathbf{w}$ and compromise rankings;
6. A root SHA-256 cryptographic checksum computed over the execution payload.

Any researcher can verify the integrity of published experimental data by reloading the JSON manifest into EmoPyLab’s audit parser, guaranteeing verifiable scientific provenance.

8. Reproducibility and Scientific Desktop Workspace

To democratize advanced evolutionary computation without requiring complex scripting, EmoPyLab provides a reactive scientific desktop workspace designed with high-contrast typography, intuitive interactive visualization, and strict memory safety guarantees.

8.1. Quick Test Diagnostic Workspace

The *Quick Test* workspace (Figure 2) provides an interactive environment for rapid algorithm prototyping and single-run diagnostic visualization:

- **Panel A (Scenario Selection Catalog):** Interactive tree browser organizing 298+ metaheuristics and 916 problem formulations into searchable categories with real-time filtering.
- **Panel B (Review Parameters & Accelerator):** Interactive sliders and controls to calibrate population size N , maximum generations, decision dimension D , objective dimension M , and select the active hardware acceleration tier (CUDA, MLX, JAX, CuPy, or CPU SIMD).
- **Panel C (Run and Inspect 3D Visualizer):** Real-time 2D and 3D Pareto front projection with interactive camera rotation, zoom, dynamic convergence trajectory tracing, and Pareto frontier overlay.
- **Panel D (Metrics and Technical Log):** Live evaluation of IGD+, GD+, Spacing, and Fast-MC Hypervolume alongside real-time hardware execution telemetry and memory bandwidth monitoring.

8.2. Experiment Module: High-Throughput Batch Benchmarking

The *Experiment Module* (Figure 3) automates the execution and synthesis of publication-grade multi-algorithm benchmarking matrices:

- **Panel A (Benchmark Matrix Configuration):** Multi-selection matrix allowing users to configure dozens of metaheuristics against multiple benchmark test suites, objective dimensions, and evaluation budgets simultaneously.
- **Panel B (Seed Planning and Concurrency):** Deterministic seed schedule generation (Fixed, Random, or Arithmetic sequences) paired with multi-core CPU worker thread allocation and GPU worker assignment.
- **Panel C (Statistical Synthesis):** Consolidated calculation of parametric metrics (mean, standard deviation) and non-parametric metrics (median, interquartile range IQR) accompanied by automated Wilcoxon signed-rank and Friedman ranking tests [37].
- **Panel D (Publication-Ready L^AT_EX Export):** One-click generation of fully formatted L^AT_EX booktabs tables, complete with automated best-performer bolding and statistical significance symbols (+, ≈, −).

8.3. Memory-Safe Ring-Buffer Architecture

Long-running evolutionary benchmarks often suffer from memory bloat in graphical environments due to the continuous accumulation of historical generational front objects in RAM. EmoPyLab enforces strict memory safety through a circular ring-buffer architecture.

During optimization runs, population objective histories are buffered within a fixed-capacity circular memory pool $\mathcal{B} \in \mathbb{R}^{B \times N \times M}$ with $B = 100$ generational slots. As new generations execute, older snapshots are lazily downsampled and streamed to disk in compressed binary format. This design ensures that the application’s resident set size (RSS) memory footprint remains strictly bounded below 150 MB regardless of generation count or population size, preventing out-of-memory crashes during multi-day experimental campaigns.

8.4. Headless CLI Execution Engine

For supercomputing environments and automated continuous integration pipelines, all diagnostic and benchmarking capabilities are exposed via a headless Command Line Interface (CLI):

```
1 # High-throughput batch benchmark on GPU
2 python -m emopylab.cli benchmark \
3     --algorithms nsga2,nsga3,moead,rvea \
4     --problems maf1,maf2,maf3,maf4,maf5 \
5     --objectives 5,10,15 \
6     --seeds 30 \
7     --backend cuda \
8     --workers 36 \
```

```

9 | --export-latex results.tex \
10 | --manifest-dir ./manifests

```

The CLI generates identical SHA-256 sidecars and formatted L^AT_EX tables, ensuring seamless equivalence between interactive desktop exploration and distributed supercomputing clusters.

9. Conclusion and Future Directions

In this paper, we presented *EmoPyLab*, an open-source, tensor-native benchmarking and decision-making platform designed to resolve foundational computational, analytical, and operational bottlenecks in Evolutionary Multi-Objective and Many-Objective Optimization ($M \geq 2$). By grounding the internal execution model in a contiguous column-major Structure-of-Arrays (SoA) memory tensor formulation, the architecture delivers high-throughput hardware acceleration across a 5-tier backend dispatch engine spanning NVIDIA GPUs (CUDA), AMD GPUs (ROCm), Apple Silicon (MLX), Google JAX (XLA), and vectorized multicore CPU SIMD. The integration of deductive Efficient Non-dominated Sorting (ENS) and quasi-Monte Carlo Hypervolume (Fast-MC HV) with Sobol low-discrepancy sequences successfully mitigates classical $O(MN^2)$ sorting and $\#P$ -hard metric scaling bottlenecks, enabling real-time analytical evaluation in up to 15-objective search spaces.

Furthermore, EmoPyLab bridges the historical disconnect between Pareto front discovery and practical engineering deployment by embedding an in-situ Multi-Criteria Decision Making (MCDM) layer (TOPSIS, PROMETHEE II net outranking flows, and Compromise Programming) with deterministic decision-space back-mapping. The ecosystem unifies automated non-parametric inferential statistical testing (Wilcoxon, Friedman with Kendall’s W , Vargha-Delaney A_{12} , and Holm-Bonferroni corrections) with cryptographic SHA-256 reproducibility manifests and a memory-safe reactive scientific desktop workspace maintaining a constant RAM footprint (< 150 MB). Large-scale validation across 5,970 benchmark runs on the Santos Dumont Bull Sequana supercomputer demonstrated linear multi-seed scaling and up to $19.39\times$ metric acceleration.

Future research directions include extending the tensor compilation engine to automated mixed-integer and combinatorial representations, incorporating interactive reinforcement learning surrogates directly within the device memory loop, and deploying distributed multi-node JAX mesh parallelism across federated supercomputing clusters.

Acknowledgment

The authors express their sincere gratitude to the National Laboratory for Scientific Computing (LNCC/MCTI, Brazil) for providing high-performance computing resources on the Santos Dumont Supercomputer (SDumont). The authors also thank the METISBr research group (<https://github.com/METISBR>) for constructive discussions and collaborative testing.

References

- [1] C. A. Coello Coello, S. González Brambila, J. Figueroa Gamboa, M. G. Castillo Tapia, R. Hernández Gómez, Evolutionary multiobjective optimization: open research areas and some challenges lying ahead, *Complex & Intelligent Systems* 6 (1) (2020) 1–24, doi: <https://doi.org/10.1007/s40747-019-0113-4>. doi:<https://doi.org/10.1007/s40747-019-0113-4>.
- [2] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*, John Wiley & Sons, 2001.
- [3] H. Ishibuchi, N. Tsukamoto, Y. Nojima, Evolutionary many-objective optimization: A short review, in: *IEEE Congress on Evolutionary Computation (CEC)*, 2008, pp. 2419–2426.
- [4] B. Li, J. Li, K. Tang, X. Yao, Many-objective evolutionary algorithms: A survey, *ACM Computing Surveys* 48 (1) (2015) 1–35.
- [5] M. Farina, P. Amato, On the positive definition of pareto optimality in many-objective optimization, *IEEE Transactions on Evolutionary Computation* 8 (6) (2004) 650–668.
- [6] Q. Zhang, H. Li, Moea/d: A multiobjective evolutionary algorithm based on decomposition, *IEEE Transactions on Evolutionary Computation* 11 (6) (2007) 712–731, doi: <https://doi.org/10.1109/TEVC.2007.892759>. doi:<https://doi.org/10.1109/TEVC.2007.892759>.
- [7] K. Deb, H. Jain, An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting: Nsga-iii, *IEEE Transactions on Evolutionary Computation* 18 (5) (2014) 719–731, doi: <https://doi.org/10.1109/TEVC.2013.2281535>. doi:<https://doi.org/10.1109/TEVC.2013.2281535>.
- [8] J. Bader, E. Zitzler, Hype: An algorithm for fast hypervolume-based many-objective optimization, *Evolutionary Computation* 19 (1) (2011) 45–76.
- [9] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, V. G. da Fonseca, S-metric selection: Evolutionary multiobjective optimization based on dominated hypervolume, *European Journal of Operational Research* 143 (2) (2002) 497–511, doi: [https://doi.org/10.1016/S0377-2217\(02\)00275-2](https://doi.org/10.1016/S0377-2217(02)00275-2). doi:[https://doi.org/10.1016/S0377-2217\(02\)00275-2](https://doi.org/10.1016/S0377-2217(02)00275-2).
- [10] R. Cheng, Y. Jin, M. Olhofer, B. Sendhoff, Reference vector guided evolutionary algorithm for many-objective optimization, *IEEE Transactions on Evolutionary Computation* 20 (5) (2016) 773–791, doi: <https://doi.org/10.1109/TEVC.2016.2519378>. doi:<https://doi.org/10.1109/TEVC.2016.2519378>.

- [11] K. Deb, L. Thiele, M. Laumanns, E. Zitzler, Scalable multi-objective optimization test problems, in: *Proceedings of the 2002 IEEE Congress on Evolutionary Computation*, IEEE, 2002, pp. 825–832, doi: <https://doi.org/10.1109/CEC.2002.1007032>. doi:<https://doi.org/10.1109/CEC.2002.1007032>.
- [12] S. Huband, P. Hingston, L. Barone, L. While, A review of multiobjective test problems and a scalable test problem toolkit, *IEEE Transactions on Evolutionary Computation* 10 (5) (2006) 477–506.
- [13] R. Cheng, M. Li, Y. Tian, X. Zhang, S. Yang, Y. Jin, X. Yao, A benchmark test suite for evolutionary many-objective optimization, *Complex & Intelligent Systems* 3 (1) (2017) 67–81.
- [14] Y. Tian, R. Cheng, X. Zhang, Y. Jin, Platemo: A matlab platform for evolutionary multi-objective optimization, *IEEE Computational Intelligence Magazine* 12 (4) (2017) 73–87, doi: <https://doi.org/10.1109/mci.2017.2742868>. doi:<https://doi.org/10.1109/mci.2017.2742868>.
- [15] J. Blank, K. Deb, pymoo: Multi-objective optimization in python, *IEEE Access* 8 (2020) 89497–89509, doi: <https://doi.org/10.1109/access.2020.2990567>. doi:<https://doi.org/10.1109/access.2020.2990567>.
- [16] A. Benítez-Hidalgo, A. J. Nebro, J. García-Nieto, I. Oregi, J. Del Ser, jmetalpy: A python framework for multi-objective optimization with metaheuristics, *Swarm and Evolutionary Computation* 51 (2019) 100598, doi: <https://doi.org/10.1016/j.swevo.2019.100598>. doi:<https://doi.org/10.1016/j.swevo.2019.100598>.
- [17] B. Huang, R. Cheng, Z. Li, Y. Jin, K. C. Tan, Evox: A distributed gpu-accelerated framework for scalable evolutionary computation, *IEEE Transactions on Evolutionary Computation* 28 (4) (2024) 1000–1014. doi:<https://doi.org/10.1109/TEVC.2024.3389025>.
- [18] L. While, L. Bradstreet, L. Barone, A fast way of calculating exact hypervolumes, *IEEE Transactions on Evolutionary Computation* 16 (1) (2012) 86–95. doi:<https://doi.org/10.1109/TEVC.2010.2077298>.
- [19] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, C. Gagné, DEAP: Evolutionary algorithms made easy, *Journal of Machine Learning Research* 13 (2012) 2171–2175.
- [20] J. J. Durillo, A. J. Nebro, jmetal: A java framework for multi-objective optimization, *Advances in Engineering Software* 42 (10) (2011) 760–771, doi: <https://doi.org/10.1016/j.advengsoft.2011.05.014>. doi:<https://doi.org/10.1016/j.advengsoft.2011.05.014>.

- [21] G. Misitano, B. S. Saini, B. Afsar, B. Shavazipour, K. Miettinen, Desdeo: The modular and open source framework for interactive multiobjective optimization, *IEEE Access* 9 (2021) 148277–148295, doi: <https://doi.org/10.1109/access.2021.3123825>. doi:<https://doi.org/10.1109/access.2021.3123825>.
- [22] F. Liu, X. Lin, Z. Wang, S. Yao, X. Tong, M. Yuan, Q. Zhang, Large language model for multi-objective evolutionary optimization, *arXiv preprint* doi: <https://doi.org/10.48550/arXiv.2310.12541> (2024). arXiv:2310.12541, doi:<https://doi.org/10.48550/arXiv.2310.12541>.
- [23] R. Guo, Structure-aware multi-stage adaptation of qwen-72b for code generation, in: *Proceedings of the 2025 International Conference on Artificial Intelligence, Virtual Reality and Interaction Design*, 2026, p. 411–417, doi: <https://doi.org/10.1145/3777730.3777795>. doi:<https://doi.org/10.1145/3777730.3777795>.
- [24] M. C. Aytekin, F. G. Calli, M. U. Demirezen, Automating code generation for a new ecosystem: Establishing baselines with large language model based code generation for arkts and harmonyos, *Research Square* doi: <https://doi.org/10.21203/rs.3.rs-7362986/v1> (2025). doi:<https://doi.org/10.21203/rs.3.rs-7362986/v1>.
- [25] Z. Janković, B. Vesin, Clean genetic algorithm architecture for improved modularity and extensibility, in: *2023 4th International Conference on Communications, Information, Electronic and Energy Systems (CIEES)*, 2023, pp. 1–4, doi: <https://doi.org/10.1109/ciees58940.2023.10378827>. doi:<https://doi.org/10.1109/ciees58940.2023.10378827>.
- [26] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: Nsga-ii, *IEEE Transactions on Evolutionary Computation* 6 (2) (2002) 182–197, doi: <https://doi.org/10.1109/4235.996017>. doi:<https://doi.org/10.1109/4235.996017>.
- [27] K. Li, K. Deb, Q. Zhang, S. Kwong, Efficient non-dominated sorting approaches for many-objective optimization, *IEEE Transactions on Evolutionary Computation* 18 (4) (2014) 597–614. doi:<https://doi.org/10.1109/TEVC.2013.2280650>.
- [28] I. M. Sobol’, On the distribution of points in a cube and the approximate evaluation of integrals, *USSR Computational Mathematics and Mathematical Physics* 7 (4) (1967) 86–112.
- [29] H. Ishibuchi, N. Tsukamoto, Y. Nojima, Performance metrics in multiobjective optimization: A unified framework and new definitions, *IEEE Transactions on Evolutionary Computation* 20 (1) (2016) 86–98, doi: <https://doi.org/10.1109/TEVC.2015.2432813>. doi:<https://doi.org/10.1109/TEVC.2015.2432813>.

- [30] X. Zhang, X. Lin, Y. Zhang, Y. Chen, Q. Zhang, Umoea/d: A multi-objective evolutionary algorithm for uniform pareto objectives based on decomposition, arXiv preprintDoi: <https://doi.org/10.48550/arXiv.2402.09486> (2024). doi:<https://doi.org/10.48550/arXiv.2402.09486>.
- [31] Y. Zhang, S. Ding, A. Zhou, Adaptive decomposition-based evolutionary algorithm for many-objective optimization, IEEE Transactions on Cybernetics 50 (8) (2020) 3747–3760, doi: <https://doi.org/10.1109/TCYB.2019.2932855>. doi:<https://doi.org/10.1109/TCYB.2019.2932855>.
- [32] T. H. Chang, S. M. Wild, Parmoo: A python library for parallel multiobjective simulation optimization, Journal of Open Source Software 8 (82) (2023) 4468, doi: <https://doi.org/10.21105/joss.04468>. doi:<https://doi.org/10.21105/joss.04468>.
- [33] P. Chen, J. Liang, K. Qiao, P. N. Suganthan, X. Ban, A two-stage evolutionary framework for multi-objective optimization, arXiv preprintDoi: <https://doi.org/10.48550/arXiv.2407.06536> (2024). doi:<https://doi.org/10.48550/arXiv.2407.06536>.
- [34] C.-L. Hwang, K. Yoon, Multiple Attribute Decision Making: Methods and Applications, Springer-Verlag, New York, NY, USA, 1981.
- [35] J.-P. Brans, P. Vincke, Note—A field guide to PROMETHEE methods, Management Science 31 (6) (1985) 647–656.
- [36] M. Zeleny, Compromise Programming, University of South Carolina Press, Columbia, SC, USA, 1973.
- [37] J. Carrasco, S. García, M. M. Rueda, S. Das, F. Herrera, Recent trends in the use of statistical tests for comparing swarm and evolutionary computing algorithms: Practical guidelines and a critical review, Swarm and Evolutionary Computation 54 (2020) 100665. doi:<https://doi.org/10.1016/j.swevo.2020.100665>.
- [38] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.
- [39] M. Friedman, The use of ranks to avoid the assumption of normality implicit in the analysis of variance, Journal of the American Statistical Association 32 (200) (1937) 675–701.
- [40] A. Vargha, H. D. Delaney, A critique and modification of the CL common language effect size indicator of McGraw and Wong, Journal of Educational and Behavioral Statistics 25 (2) (2000) 101–132.
- [41] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

- [42] C. Collberg, T. Proebsting, G. Moraila, Repeatability in computer systems research, *Communications of the ACM* 59 (3) (2016) 62–69, doi: <https://doi.org/10.1145/2812803>. doi:<https://doi.org/10.1145/2812803>.